Blockchain Experiment 1

Name: Atharva Lotankar
Class: D20C **R.No.:** 20
Date: 12 / 08 / 2026

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Tasks Performed:

1. **Hash Generation using SHA-256:**
   - Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.
2. **Target Hash Generation with Nonce:**
   - Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. **Proof-of-Work Puzzle Solving:**
   - Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.
4. **Merkle Tree Construction:**
   - Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program #1: Python program that uses the hashlib library to create the hash of a given string:

Program:

```
import hashlib
def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string =  hash_object.hexdigest()
    # Return the hash string
    return hash_string
```

```
# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)
```

Output:

```
Enter a string: Atharva
Hash: 51056b6a6a7b468483de6de32b530b482ab397dc83ec34c2ebdcf24bf6b4321d
```

Program #2: Program to generate required target hash with input string and nonce

Program:

```
import hashlib
# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)
```

Output:

```
Enter a string: Blockchain
Enter the nonce: 20
Hash Code: b4fba819ca27a6ad7091d9a3b02d5e5118a9b7e0158b54fe973c479703d342cf
```

Program #3: Python code for Solving Puzzle for leading zeros with expected nonce and given string

Program:

```
import hashlib
def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros
    while True:
```

```
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
            print("Hash:",hash_code )
            return nonce

        nonce += 1

# Get user input
input_string = "Atharva"
num_zeros = 5

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)
```

Output:

```
Hash: 00000935de79b5d4e83aa4f186235b24a826c8dff3122b789340ae0b5ebd7cb3
Input String: Atharva
Leading Zeros: 5
Expected Nonce: 202131
```

Program 4: Generating Merkle Tree for given set of Transactions

- Update the transactions list with valid entries.
  Eg : transactions = [‘A->B: $10’, ‘B->C : $5’, ‘C->A : $2’]
- Sample Transactions to be considered
  T1 : Alice → Bob : $200; T2 : Bob → Dave : $500; T3 : Dave → Eve : $100; T4 : Eve → Alice : $300; T5 : Roo → Bob : $50
- Hash the transactions before combining them in the for loop
- Print all the intermediate hash during the construction of the Merkle Tree Root Hash

Program:

```
import hashlib
def sha256_hash(data):
    return hashlib.sha256(data.encode('utf-8')).hexdigest()

def build_merkle_tree(transactions):
    if len(transactions) == 0:
        return None

    # Step 1: Hash each transaction before building the tree
    transactions = [sha256_hash(tx) for tx in transactions]

    print("Initial Transaction Hashes:")
    for i, tx_hash in enumerate(transactions, 1):
        print(f"T{i}: {tx_hash}")

    # Step 2: Construct the Merkle Tree
    level = 1
    while len(transactions) > 1:
        print(f"\nLevel {level} Hashes:")

        # If odd number of hashes, duplicate the last hash
        if len(transactions) % 2 != 0:
            transactions.append(transactions[-1])
        new_transactions = []
        for i in range(0, len(transactions), 2):
            # Combine two hashes
            combined = transactions[i] + transactions[i + 1]
            # Hash the combined value
            hash_combined = sha256_hash(combined)
            print(f"Hash {i // 2 + 1}: {hash_combined}")
            new_transactions.append(hash_combined)
        transactions = new_transactions
        level += 1
    return transactions[0]
# Sample Transactions
transactions = [
    "Atharva -> Manaswi : $520",
    "Manaswi -> Ronit : $100",
    "Ronit -> Shakti : $256",
    "Shakti -> John : $300",
    "John -> Manaswi : $95"
```

```
]
# Construct the Merkle Root
merkle_root = build_merkle_tree(transactions)
print("\nMerkle Root:", merkle_root)
```

Output:

```
Initial Transaction Hashes:
T1: d33d1f5acaebcae17abd1f540d93adb3b61e9423ced74facac1abad01d7ca6ed
T2: ca4b33103b15e11aaa0d4091ab028738efd2342c1dd8e9a32f430d2b05bf575d
T3: 5a76364d4b59a83bdd4e3187e4b5fdf06a578ef6e6c9ccb2a79d7f1b9758749c
T4: 2e75f28298e96fc2bbb20d9d5e550d405c72be12a2b43f465735c1368082f432
T5: b50853caf6743b5af1feb6a3428d0785469c28181870f96516624781f55b91d8

Level 1 Hashes:
Hash 1: 436c5e3d17c4a6e95a602d3aa890dbd651a95523ce3cbdbdb85c3e8cbacd441e
Hash 2: fb5f9310a5e6b5cb9105cc62688f583e445e127483357b70714767a1c0aa0b3f
Hash 3: 470eddda977e12d3fa4c3795d397b2c9579f41af661540708f1c8a17e5a2c77b

Level 2 Hashes:
Hash 1: ff4871901b3721c1773d523e2bf6af12341787981070497d3ae8e49de0152250
Hash 2: 4a4e73c68889ec59744496dae175e70cc0919e5197a861ae5f9c642629137aa9

Level 3 Hashes:
Hash 1: 1633df9eb5cab872815c416e7c4c631208be4f0bcc07e91a7b4cb5c241594d03

Merkle Root: 1633df9eb5cab872815c416e7c4c631208be4f0bcc07e91a7b4cb5c241594d03
```

Conclusion: This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- **Proof-of-Work Puzzle** Solving shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.